WebSocket & Socket.IO — Complete Cheat Sheet

Roman Urdu Training Guide · Basic → Advance · 30 Practical Tasks · Full-Stack Developer Track

> **Kaise parhna hai:** Part 0 mein samjho ke HTTP aur WebSocket farq kya hai (ye concept hai, yaad rakho). Part 1 = raw WebSocket cheat sheet (native API + Node `ws`). Part 2 = Socket.IO cheat sheet (real apps mein zyada use yahi hota hai). Part 4 = dono ka farq + kab kaunsa choose karna hai. Phir **Part 5 ke 30 practical tasks** banao — yahi asal learning hai. Part 6 ka plan follow karo, Part 7 se interview ki tayari karo. Rule: **har task kam az kam do baar chalao — ek baar raw WebSocket se, ek baar Socket.IO se.**

Part 0 — Foundation: Pehle ye samjho

Concept	Easy explanation
HTTP (normal web)	Client poochta hai → Server jawab deta hai → connection band. *Request–Response*, one-way, har baar naya round trip. Live chahiye to baar baar polling (refresh) karni parti hai.
WebSocket	Ek baar connection khulta hai (handshake), phir **don taraf se** (bidirectional) message ja sakte hain — connection tab tak khula rehta hai. Real-time apps ka foundation: chat, live scores, notifications, collaborative editors.
Handshake / Upgrade	Shuru mein HTTP request jaati hai headers ke saath `Upgrade: websocket` + `Connection: Upgrade` ke saath. Server `101 Switching Protocols` bheje to connection WebSocket ban jata hai.
ws:// vs wss://	ws = port 80 (plain), **wss = port 443 (TLS/SSL encrypted)** — production mein hamesha wss (jaise http vs https).
Socket.IO kya hai?	WebSocket par bana **library/framework** — named events, rooms, ack, auto-reconnection, fallback (long-polling) jaise features ready-made. "WebSocket ka high-level wrapper."
SSE (option)	Server → Client sirf ek taraf (text). Automatic reconnect hota hai. Agar sirf server se live feed bhejni ho (prices/notifications) to SSE bhi option hai; do taraf chahiye to WebSocket.

> **Kab WebSocket?** Jab data ki frequency high ho ya user ko bhi bhejna ho: chat, live bidding, multiplayer game, live dashboard, typing indicator, collaborative docs. **Kab nahi?** Simple CRUD APIs — wahan REST/HTTP hi behtar hai (stateless, cacheable, simple).

WebSocket handshake kaise dikhta hai

```
GET /chat HTTP/1.1
Host: example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: x3JJHMbDL1EzLkh9GBhXDw==
Sec-WebSocket-Version: 13

→ Jawab: HTTP/1.1 101 Switching Protocols
  (ab se dono taraf WebSocket frames — HTTP nahi)
```

Setup (easy way — abhi practice ke liye)

```
mkdir ws-lab && cd ws-lab && npm init -y
npm install ws socket.io socket.io-client
# debugging client (CLI):  npx wscat -c ws://localhost:8080
# online practice: socket.io mein "get started" demo chala lo
```

Part 1 — WebSocket Cheat Sheet (RAW / Native)

1.1 Browser Client — native WebSocket API

```
const ws = new WebSocket("wss://example.com/ws");

ws.onopen = () => {
  console.log("connected");
  ws.send(JSON.stringify({ type: "chat", text: "Hello!" }));
};

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);   // JSON khud parse karna parta hai
  console.log("server says:", data);
};

ws.onclose = (event) => {
  console.log("closed", event.code, event.reason);
  // yahan reconnect logic likho (neeche 1.5)
};

ws.onerror = (err) => console.error("error", err);

// Bhejne se pehle state check (optional)
if (ws.readyState === WebSocket.OPEN) ws.send("hi");

ws.close(1000, "user left");   // clean close
```

1.2 Connection states (readyState) aur Close codes

readyState	Meaning
0 — CONNECTING	Handshake ho raha hai
1 — OPEN	Connection ready — send/recv chalu
2 — CLOSING	close() call hua, band ho raha hai
3 — CLOSED	Band — ab message nahi jayega

Code	Meaning	Kab milta hai
1000	Normal Closure	Neat tareeqe se close (close() call)
1001	Going Away	Server/page band ho raha hai
1006	Abnormal Closure	**Connection toot gaya** (network drop, server crash) — close frame nahi aaya
1008	Policy Violation	Rules tora (auth/size)
1009	Message Too Big	Message size limit se zyada
4000–4999	App-level codes	Khud define karo (jaise 4001 = unauthorized)

1.3 Node.js Server — `ws` library

```
import { WebSocketServer } from "ws";

const wss = new WebSocketServer({ port: 8080 });

wss.on("connection", (socket, req) => {
  console.log("new client:", socket.id ?? req.socket.remoteAddress);

  socket.on("message", (raw) => {
    // raw = Buffer — JSON parse karo
    let msg;
    try { msg = JSON.parse(raw.toString()); }
    catch { return socket.send(JSON.stringify({ type: "error", text: "bad json" })); }

    // Echo: wahi wapas
    socket.send(JSON.stringify({ type: "echo", text: msg.text }));

    // Broadcast: sab clients ko
    wss.clients.forEach((client) => {
      if (client !== socket && client.readyState === 1) {
        client.send(JSON.stringify({ type: "chat", text: msg.text }));
      }
    });
  });

  socket.on("close", (code) => console.log("closed", code));
  socket.on("error", (err) => console.error(err));
});

// Dead clients hatane ka heartbeat (server-side ping — browser yeh nahi kar sakta)
setInterval(() => {
  wss.clients.forEach((s) => {
    if (!s.isAlive) return s.terminate();   // 2 dafa ping nahi mila = mar gaya
    s.isAlive = false;
    s.ping();
  });
}, 30000);
wss.on("connection", (s) => { s.isAlive = true; s.on("pong", () => (s.isAlive = true)); });
```

1.4 Raw WebSocket — handy table

Kaam	Browser (client)	Node `ws` (server)
Connect	`new WebSocket(url)`	`new WebSocketServer({ port })`
Send	`ws.send(data)`	`socket.send(data)`
Receive	`ws.onmessage`	`socket.on("message", cb)`
Ping/Pong	Browser API mein nahi — app-level heartbeat banao	`socket.ping()` / `socket.on("pong")`
Binary data	`ws.binaryType = "arraybuffer"`	Buffer aata hai default
Sab clients	—	`wss.clients` (Set)
Close	`ws.close(1000, "bye")`	`socket.close()` / `socket.terminate()`
Message format	String (JSON khud stringify/parse karo) ya binary — koi event name nahi, **khud protocol banana parta hai**	

1.5 Client reconnect (exponential backoff) — raw mein khud likhna parta hai

```
function connect(url, onMsg) {
  const ws = new WebSocket(url);
  let tries = 0;
  ws.onopen = () => { tries = 0; };
  ws.onmessage = onMsg;
  ws.onclose = () => {
    const delay = Math.min(30000, 1000 * 2 ** tries++);  // 1s, 2s, 4s... 30s cap
    setTimeout(() => connect(url, onMsg), delay);
  };
}
connect("wss://example.com/ws", (e) => console.log(e.data));
```

Part 2 — Socket.IO Cheat Sheet

> **Install:** server → `npm i socket.io` · client → `npm i socket.io-client` (browser mein CDN bhi: `<script src="/socket.io/socket.io.js">`). **Version:** v4.x (v3/v4 mein breaking changes — purane tutorials ka dhyan).

2.1 Server

```
import { createServer } from "http";
import { Server } from "socket.io";

const httpServer = createServer();
const io = new Server(httpServer, {
  cors: { origin: "http://localhost:3000", methods: ["GET","POST"] },
  // transports: ["websocket"],      // polling skip karna ho to
  // path: "/socket.io",             // default
  // pingInterval: 25000, pingTimeout: 20000,   // defaults (ms)
  // maxHttpBufferSize: 1e6,         // 1MB — bade message se bachne ke liye
});

// ---- Auth middleware (handshake ke waqt) ----
io.use((socket, next) => {
  const token = socket.handshake.auth?.token;      // headers bhi milte hain
  if (!verify(token)) return next(new Error("unauthorized"));
  socket.data.userId = decode(token).id;           // aage ke handlers ke liye
  next();
});

io.on("connection", (socket) => {
  console.log("connected:", socket.id);
  socket.join(`user:${socket.data.userId}`);       // personal room

  // ---- Ek client se aaya ----
  socket.on("chat:message", (data, ack) => {
    if (!data?.text) return ack?.({ ok: false, error: "empty" });
    io.to("lobby").emit("chat:message", { from: socket.id, text: data.text, at: Date.now() });
    ack?.({ ok: true });                           // sender ko confirmation
  });

  // ---- Sab ko bhejna (tarika yaad rakho) ----
  // io.emit(...)                  = poori default namespace sabko
  // socket.emit(...)              = sirf yehi ek client
  // socket.broadcast.emit(...)    = sabko chhod kar (ye wala client nahi)
  // io.to("lobby").emit(...)      = sirf lobby room ke sab members
  // socket.to("lobby").emit(...)  = lobby ke members, sender ko chhod kar

  socket.on("disconnecting", (reason) => {
    console.log("rooms before leave:", [...socket.rooms]);   // leave se pehle
  });
  socket.on("disconnect", (reason) => console.log("gone:", reason));
});

httpServer.listen(4000);
```

2.2 Client

```
import { io } from "socket.io-client";

const socket = io("http://localhost:4000", {
  auth: { token: localStorage.getItem("token") },
  // transports: ["websocket"],
  // reconnection: true, reconnectionDelay: 1000, reconnectionAttempts: Infinity  (defaults)
});

socket.on("connect", () => {
  console.log("id:", socket.id);
  socket.emit("join", { room: "lobby" });
});

socket.on("chat:message", (msg) => appendToUI(msg));
socket.on("connect_error", (err) => console.error(err.message));   // auth fail yahan
socket.on("disconnect", (reason) => {
  // reason: "transport close" | "ping timeout" | "io server disconnect" (= server ne
  // disconnect() call kiya — CLIENT DOBARA CONNECT NAHI KAREGA, khud dobara connect karo)
});

// Ack (2-way confirmation) + timeout
socket.timeout(3000).emit("chat:message", { text: "hi" }, (err, res) => {
  if (err) return console.log("no answer in 3s");
  console.log("server ack:", res);
});

socket.disconnect();
```

2.3 Rooms, Namespaces, Presence — cheat table

Kaam	Code
Room join / leave	`socket.join("lobby")` · `socket.leave("lobby")` (server par)
Room ke members	`io.sockets.adapter.rooms.get("lobby")` → Set of socket ids
Room mein bhejna	`io.to("lobby").emit(...)` (sab) · `socket.to("lobby").emit(...)` (sender chhod kar)
Kya room hoon main?	`socket.rooms` → Set (apna id hamesha hota hai)
Namespace = alag pipeline	`io.of("/admin")` · client: `io("/admin")` — default namespace `"/"` ; events/rooms alag rehte hain
Specific user ko bhejna	Har user ki personal room `user:<id>` banao → `io.to("user:" + uid).emit(...)`
Volatile (drop allowed)	`socket.volatile.emit("cursor", pos)` — connected na ho to *drop*, queue nahi (live cursor/typing ke liye)
Binary message	`socket.emit("file", buffer)` — Buffer/Blob dono chalte hain
Server se client band karna	`socket.disconnect(true)` — client ko "io server disconnect" milega
Connected kitne clients	`io.engine.clientsCount`

2.4 Important options

Option	Default	Kab badlo
`transports`	["polling","websocket"]	Pehle HTTP long-polling, phir WS par upgrade. Agar proxy/pipeline issue ho ya sirf WS chahiye → `["websocket"]`
`path`	/socket.io	Proxy/URL conflict ho to

`pingInterval / pingTimeout`	25000 / 20000 ms	Dead client jaldi detect karna ho (Socket.IO khud heartbeat karta hai)
`maxHttpBufferSize`	1e6 (1MB)	DoS/bade payload se bachne ke liye chhota rakho
`cors.origin`	—	Hamesha apna frontend origin do — wildcard mat lagao production mein
Client: `reconnection`	true	Auto reconnect + backoff built-in (raw mein khud likhna parta tha)
Client: `auth`	—	Handshake mein token bhejo (query string mein URL mein mat bhejo — logs mein reh jata hai)

2.5 Events ka poora map (yaad rakho)

Kahan	Events
Server — connection	`io.on("connection", socket)` · errors ke liye `io.use(middleware)`
Server — socket lifecycle	`disconnecting` (rooms chhodne se pehle) · `disconnect` (baad) · custom: jo bhi `socket.on("...")`
Client	`connect` · `connect_error` · `disconnect(reason)`
Lifecycle achi tarah	Reconnect par **rooms dobara join** karo aur missed messages sync karo — client events connection state par depend karte hain

2.6 Debugging

```
DEBUG=socket.io,engine.io node server.js        # server ka detailed log
npx wscat -c ws://localhost:4000/socket.io/?EIO=4&transport=websocket   # low-level peek
# Browser: DevTools → Network → WS tab → frames live dekho
# Admin UI: npm i @socket.io/admin-ui → io.attachAdminUI() → real-time monitoring
```

Part 3 — Common Patterns (production mein yehi chalte hain)

Pattern 1 — Chat message ka flow

```
Client emit "chat:message" → Server validate (zod) → DB save →
io.to(room).emit("chat:message", savedMsg) → sab clients render → sender ko ack
```

Pattern 2 — Typing indicator (debounce + volatile)

```
// Client: input event par (500ms debounce ke baad)
socket.volatile.emit("typing", { room, user });   // drop ho jaye to chalega
// Server: socket.to(room).emit("typing", { user }) — sender ko chhod kar
// Client: 2 sec baad "stopped:typing" ya timeout se UI hatao
```

Pattern 3 — Presence (kaun online hai)

```
// Server (memory ya Redis):
const online = new Map();   // userId → socketId
io.on("connection", (s) => {
  online.set(s.data.userId, s.id);
  io.emit("presence:update", [...online.keys()]);
  s.on("disconnect", () => { online.delete(s.data.userId); io.emit("presence:update", [...online.keys()]); });
});
```

Pattern 4 — React mein Socket.IO (custom hook)

```
import { useEffect, useState } from "react";
import { io } from "socket.io-client";

export function useSocket(url, token) {
  const [socket, setSocket] = useState(null);
  useEffect(() => {
    const s = io(url, { auth: { token } });
    setSocket(s);
    return () => { s.disconnect(); };     // cleanup ZAROORI (StrictMode double-mount!)
  }, [url, token]);
  return socket;
}

// Usage: const socket = useSocket(NEXT_PUBLIC_WS_URL, token);
// useEffect(() => { if (!socket) return; socket.on("chat:message", h); return () => socket.off("chat:message", h);
}, [socket]);
```

> **Next.js dhyan:** Next.js API routes stateless aur serverless hain — long-lived WebSocket connection wahan nahi chalti. Options: (1) custom `server.js` (http + Next handler + Socket.IO), ya (2) alag WS microservice/host — modern stack mein (2) simpler rehta hai. Client components mein hi socket use karo (browser API server par nahi).

Pattern 5 — Delivery guarantee ka soch (reconnect + history)

```
Rule: disconnect ke duran ke messages drop ho sakte hain. Hamesha:
1) Har message ko DB/queue mein persist karo (source of truth)
2) Client reconnect (socket.on("connect")) par "sync" emit karo: { since: lastTs }
3) Server missed messages bheje — UI merge kare
Aur UI mein dikhao: "reconnecting..." (socket.on("disconnect")) jab tak message safe nahi.
```

Part 4 — WebSocket vs Socket.IO (ye farq interview mein poocha jata hai)

Point	WebSocket (raw)	Socket.IO
Kya hai	W3C standard **protocol/API** — browser ka native API	Library — WebSocket (aur polling) ke upar bana **framework**
Connection	Sirf WebSocket + manual HTTP upgrade	Pehle HTTP long-polling → phir WebSocket par auto-upgrade (fallback built-in — purane proxies/networks bhi chalte hain)
Message style	Sirf `message` event — **khud protocol banana parta hai** (JSON mein type field)	**Named events**: `socket.on("chat:message")` — readable, maintainable
Reconnection	Khud likhna parta hai (backoff, retry, state)	Built-in automatic (delay, attempts, jitter)
Ack (confirmation)	Khud banana parta hai (id + callback map)	Built-in: `emit(ev, data, cb)` + `timeout()`
Rooms / Namespaces	Nahi — apna routing banao	Built-in: `join`, `io.to()`, `of("/ns")`
Auth / Middleware	Handshake mein headers/phir query — khud check karo	`io.use()` middleware + `handshake.auth`
Heartbeat	Server-side ping/pong khud lagao (`ws` mein); browser ping nahi bhej sakta	Engine.IO khud ping (25s) / timeout (20s) karta hai
Overhead	**Minimum** — sirf WS frames	Chhota JSON envelope + polling phase mein overhead; high-frequency (1000s msg/sec) par raw behtar
Debug	DevTools WS tab mein frames saaf dikhte hain	Engine.IO layer + DEBUG logging chahiye
Features extra	Sirf transport	Volatile events, binary, admin UI, adapters (Redis), versioned protocols
Learning curve	Chhota API, lekin protocol + resilience khud	Zyada concepts, par app-level kaam ready

> **Kab kaunsa?** · 90% real apps (chat, notifications, dashboards, social feed) → **Socket.IO** (features + kam code). · Simple/high-frequency (live cursor, game ticks, market ticks), minimal overhead chahiye, ya kisi SDK/client restriction hai → **raw WebSocket**. · Interview ka safe answer: "Socket.IO ek abstraction hai — raw WebSocket support karta hai plus fallback/reconnection/rooms, toh productivity ke liye; performance-critical bare-metal layer chahiye to raw ws."

Engine.IO layer (Socket.IO ke andar kya hota hai) — 1 line

```
App events  →  Socket.IO Parser  →  Engine.IO (transport: polling ⇄ websocket)  →  TCP
# socket.io = events/rooms/ack    engine.io = connection/heartbeat/upgrade    ws = frames
```

Part 5 — Practical Tasks: Basic → Advance (30)

Rules: Har task pehle *bina dekhe* likho. Hint do baad mein. Har task do baar chalao — **ek baar raw `ws` se, ek baar Socket.IO se** (jahan applicable ho). Terminal mein ek side `node server.js`, browser/test-client dusri side.

Level 1 — Basic (Week 1) 10 tasks

T1. Echo server banao (port 8080, `ws` library) — jo client bheje, server wahi wapas bheje.

Hint: `new WebSocketServer({port:8080})` → `connection` → `socket.on("message", d => socket.send(d))`. Test: `npx wscat -c ws://localhost:8080`

T2. Browser page banao jo server se connect kare, `onopen/onmessage/onclose` par console.log kare, aur ek button se text bheje.

Hint: `new WebSocket("ws://localhost:8080")` · DevTools → Network → WS tab mein frames check karo.

T3. JSON protocol banao: client `{"type":"chat","text":"..."}` bheje, server validate kare aur `{"type":"error"}` bheje agar JSON invalid ya type galat ho.

Hint: `JSON.parse` try/catch + `msg.type === "chat"` check — raw WebSocket mein message format **tum khud define karte ho**.

T4. readyState observe karo: connect hone se pehle, open hone par, `close()` ke baad — 4 states console par. Server band karke close code **1006** bhi dekho.

Hint: `ws.readyState` (0,1,2,3) · `onclose` mein `event.code` log karo.

T5. Broadcast: 2 browser tabs connect karo — ek ke message dusri tab mein dikhe (server par sab clients par loop).

Hint: `wss.clients.forEach` + `readyState === 1` check + sender ko chhod do (ya dono ko bhejo, choice tumhari).

T6. Socket.IO server + client (CDN ya npm) lagao — `emit("hello", data)` aur `on("hello")` se message bhejo. Raw ke `send/onmessage` se farq note karo.

Hint: server: `new Server(httpServer)` · client: `const s = io("http://localhost:4000")` — farq: **named event**, JSON khud parse nahi karna.

T7. Acknowledgement lagao: client emit kare *callback ke saath*, server us callback se jawab de "saved!". Client par print karo.

Hint: `socket.emit("save", data, (res) => ...)` · server: `socket.on("save", (data, ack) => ack({ok:true}))`

T8. Room banao: client `join("lobby")` kare, server `io.to("lobby").emit(...)` se sirf lobby members ko bheje. 3 clients — 1 join, 2 nahi — fark dekho.

Hint: server par `socket.join("lobby")` (custom "join" event banao) · members: `io.sockets.adapter.rooms.get("lobby")`

T9. Presence: server par online users ka `Set` — connection par add, disconnect par remove, har change par sabko updated list bhejo.

Hint: `io.on("connection")` + `socket.on("disconnect")` + `io.emit("online", [...set])`

T10. Reconnect demo: Socket.IO client ke `connect / disconnect / connect_error` events par UI badge lagao ("online"/"offline") — server restart karke dekho, client khud reconnect karega.

Hint: raw WebSocket mein yeh tum manually backoff se karoge (Part 1.5) — Socket.IO mein default hai.

Level 2 — Intermediate (Week 2–3) 10 tasks

T11. Full chat room: naam login (localStorage), messages mein **from + time** ke saath sabko dikhe, Enter par bhejo. 3 tabs se test.

Hint: "join" event (room + name) → "chat:message" → `io.to(room).emit` · UI: messages array state mein.

T12. Typing indicator: input type karne par "Ali is typing..." — 500ms debounce + `volatile.emit`, 2 sec baad hat jaye.

Hint: debounce setTimeout · server: `socket.to(room).emit("typing", name)` (sender chhod kar).

T13. Private DM: personal room `user:<id>` — server kisi specific user ko DM forward kare, receiver par conversation dikhe.

Hint: connection par `socket.join(`user:${id}`)` · bhejne wala `to` field de → server `io.to(`user:${to}`).emit("dm", msg)`.

T14. Auth: JWT token handshake mein bhejo (`io(url, { auth: { token } })`), server `io.use()` mein verify kare — galat token par connection ho aur client ko `connect_error` mile.

Hint: `socket.handshake.auth.token` · `next(new Error("unauthorized"))` · query string mein mat bhejo.

T15. React integration: `useSocket` hook (Part 3.4) banao — mount par connect, unmount par disconnect. Chat component mein messages state rakho.

Hint: cleanup return karo · StrictMode (dev) mein double-mount hota hai — `s.disconnect()` zaroori · event listener bhi `socket.off` se hatana bhoolna mat.

T16. Next.js app mein Socket.IO chalao: custom `server.js` (http server + Next handler + Socket.IO) ya alag WS service — aur frontend se live message test karo.

Hint: API routes long-lived connection support nahi karte — custom server/service hi rasta hai. Client URL env variable (`NEXT_PUBLIC_WS_URL`) se lo.

T17. Messages persist karo: chat message server par DB mein save, aur room join hone par **last 50 messages** history ke taur par bhejo.

Hint: save → emit (history `socket.emit` sirf joiner ko) · order by timestamp.

T18. Reconnect sync: client `disconnect` par "reconnecting" dikhaye, dobara `connect` hone par `{ since: lastMsgTs }` emit kare, server missed messages bheje.

Hint: Part 3.5 pattern — **yeh production chat ka core logic hai**.

T19. Namespaces: `/chat` aur `/notifications` — alag handlers, alag connection (client par `io("/chat")` dono simultaneously). Events alag rehte hain — verify karo.

Hint: `io.of("/chat").on("connection", ...)` · default namespace `"/"` alag hai.

T20. Ack + timeout: "save message" emit karo `socket.timeout(3000).emit(...)` — server down ho to error callback chale, UI par "not delivered" dikhao (message list mein status: sending/sent/failed).

Hint: pehla callback argument `err` hota hai timeout par — UI mein per-message status rakho.

Level 3 — Advance (Week 4) 10 tasks

T21. Horizontal scaling: do Node instances (port 4001/4002) + `@socket.io/redis-adapter` (Redis chalao) — client A instance-1 par, client B instance-2 par, A ka message B ko pahunche.

Hint: har instance par `createAdapter(pubClient, subClient)` · `io.to("lobby").emit` cross-instance kaam karega · polling on ho to load balancer par **sticky sessions** chahiye.

T22. Dead client detect: `pingInterval/pingTimeout` samjho, server par custom timeout handler lagao (client ke socket ko `timeout` event par log/terminate). Raw `ws` mein heartbeat interval script bhi likho.

Hint: Part 1.3 wala setInterval heartbeat · Socket.IO: options set karke offline client ko "ping timeout" par chhodta hai.

T23. Load test: Node script se **100 clients** ek saath connect karo, sab ek dusre ko message bhejein (1 sec interval) — server CPU/memory/log observe karo, phir rate aadha karke compare.

Hint: loop mein `io()` call · `io.engine.clientsCount` check · 1000 clients tak try karo — bottleneck note karo.

T24. Security hardening checklist lagao: wss:// (prod), CORS origin fix, `maxHttpBufferSize` chhota, message schema validation (zod), per-socket rate limit (simple counter), auth middleware.

Hint: har incoming event par `schema.parse(data)` try/catch · rate limit: 10 msg/sec se zyada par drop.

T25. Debugging drill: `DEBUG=socket.io,engine.io node server.js` chalao, browser DevTools WS frames parho (handshake polling → upgrade), aur `@socket.io/admin-ui` se live monitoring khola.

Hint: upgrade sequence dekho: HTTP polling requests → websocket upgrade frame. Kisi bhi "connection failed" ko frames se trace karo.

T26. Graceful shutdown: server `SIGINT` (Ctrl+C) par sab clients ko `server_shutdown` emit kare, thoda ruk kar flush kare, phir `io.close()` + exit — client turant reconnect kare aur UI par message dikhe.

Hint: `process.on("SIGINT", ...)` → `io.emit(...)` → `setTimeout(() => io.close(), 500)`.

T27. Volatile vs reliable: live cursor/position updates ke liye `volatile.emit` lagao (drop allowed), aur chat messages ke liye normal emit + ack (guaranteed) — dono ka behaviour disconnect ke waqt observe karo.

Hint: volatile = queue nahi (connected na ho to gaya) — high-frequency ephemeral data ke liye perfect.

T28. Notification system: personal rooms + unread count (DB mein), offline user ka count DB se, `join` par badge sync, har naye notification par live increment + browser Notification API se popup.

Hint: `io.to(user:X)` · unread: server par counter ya SQL `COUNT(*) WHERE seen=false` · client: `Notification.requestPermission()`.

T29. Reverse proxy (nginx) ke peeche chalao: HTTP Upgrade headers config karo (`proxy_set_header Upgrade $http_upgrade; Connection "upgrade";`), WebSocket connection establish hone verify karo — aur proxy ke timeout settings par heartbeat effect samjho.

Hint: proxy_read_timeout default 60s — pingInterval us se chhota rakho warna connection katega. WSS termination bhi proxy par ho sakta hai.

T30. Capstone: Real-time collab mini-app — auth (JWT handshake) + chat rooms + private DM + typing + presence + history sync + Redis adapter (2 instances) + load test. Production checklist (Part 7) apply karo. *Ye complete ho jaye to tum real-time engineering confidently kar sakte ho.*

Hint: T11–T21 ke pieces jodne ka kaam hai — pehle schema/protocol events list banao (kaunsa event kya karega), phir code karo.

Part 6 — 30-Day Learning Plan

Days	Kya seekhna hai	Kya banao
Day 1–2	HTTP vs WebSocket, handshake, frames, ws vs wss	Part 0–1 parho; DevTools mein kisi live site ke WS frames dekho
Day 3–5	Raw client API: states, close codes, JSON protocol	T1–T4: echo server + browser client
Day 6–7	Broadcast + heartbeat + reconnect (backoff)	T5 + 2-tab chat (raw) + Part 1.5 reconnect script
Day 8–9	Socket.IO setup: events, emit/on, server/client	T6–T7: named events + ack
Day 10–12	Rooms, adapter, presence, disconnect lifecycle	T8–T10 + T11 chat room (ye week ka main project)
Day 13–14	Differences revision (Part 4) + typing/DM patterns	T12–T13; raw vs Socket.IO comparison khud likho
Day 15–17	Auth middleware, React hook, Next.js wiring	T14–T16: React chat app with JWT
Day 18–21	Persistence, history sync, namespaces, ack timeout	T17–T20: chat with DB + delivery status
Day 22–24	Scaling: Redis adapter, sticky sessions, heartbeats	T21–T22: 2 instances demo
Day 25–27	Security, debugging, load test	T23–T25: 100 clients + DEBUG logs + admin UI
Day 28–29	Production polish: shutdown, volatile, proxy, notifications	T26–T29
Day 30	Capstone + interview prep	T30 + Part 7 ke 10 sawaal khud jawab do

Practice rhythm (roz ka nizam)

- **Roz 1–1.5 ghanta:** 30 min concept + 60 min task. Terminal + 2 browser tabs = tera lab.
- **Har task do tarah se:** pehle `ws` se, phir Socket.IO se — farq body mein feel hota hai.
- **DevTools WS tab** har roz dekho — frames na parho to concept adhura rahega.
- Har feature baad mein **DB persistence + auth** add karo — "demo" se "production" ka farq yehi hai.

Part 7 — Production Checklist + Trainer Tips + Interview

12-point production checklist (deploy se pehle):

1. **wss://** hamesha (TLS), CORS sirf apna frontend origin.
2. **Auth handshake par** — `io.use()` + token verify; connection ke baad nahi.
3. **Har message validate** (zod/joi) — client par bharosa mat karo; size limit (`maxHttpBufferSize`).
4. **Heartbeat** chalu (Socket.IO default theek hai; raw mein khud lagao) — dead sockets cleanup.
5. **Scaling:** Redis adapter + (polling on ho to) sticky sessions; single instance 10k+ connections easily.
6. **Persistence:** messages DB mein — socket memory source of truth NAHI.
7. **Reconnect UX:** "reconnecting" state + missed messages sync (`since` timestamp).
8. **Rate limiting** per socket (10–20 msg/sec) — spam/abuse se bachna.
9. **Rooms ka cleanup:** disconnect par personal rooms apne aap chhodte hain; app-level counters (online set) hatao.
10. **Timeouts:** ack callbacks mein `timeout()` lagao — hang requests nahi.
11. **Observability:** connection count, message rate, error events log karo; DEBUG mode dev mein.
12. **Graceful shutdown** + load balancer health check — zero-downtime deploys.

Interview ke 10 sawaal (pehle khud jawab do)

Sawaal	Ek line answer
HTTP aur WebSocket ka farq?	HTTP = one-way request/response, connection per request; WebSocket = bidirectional persistent connection (101 upgrade ke baad)
WebSocket kaise start hota hai?	HTTP handshake with `Upgrade: websocket` headers → server `101 Switching Protocols` → TCP connection WebSocket frames mein
Socket.IO kya hai — WebSocket hai ya nahi?	Library hai jo WebSocket (aur HTTP long-polling fallback) use karti hai — upar events/rooms/ack/reconnect ka layer
ws:// aur wss://?	wss = TLS encrypted (port 443) — production mein mandatory, jaise HTTPS
Rooms vs Namespaces?	Room = ek connection ke groups (join/leave, message routing); Namespace = poora alag pipeline/socket server (apne events + rooms), default "/"
Reconnection kaise?	Socket.IO: automatic (exponential backoff, attempts); raw: khud `onclose` + setTimeout backoff + state resync
Scale kaise karoge 100k connections?	Multiple instances + Redis adapter (cross-instance emit) + sticky sessions (polling) + OS tuning (file descriptors) + LB
Auth kaise karte ho?	Handshake mein JWT (`handshake.auth`) + `io.use()` middleware verify — invalid par `next(new Error)`; URL query mein token nahi
Offline user ko message milega?	Socket se nahi — persist karo DB mein (unread/queue), next connect par history/sync se bhejo; live emit sirf connected ko
Kab raw WebSocket, kab Socket.IO?	High-frequency minimal frames/raw control chahiye → raw; app features (rooms, ack, reconnect, fallback) chahiye → Socket.IO